

Search Pipeline Architecture

Enterprise AI Companion

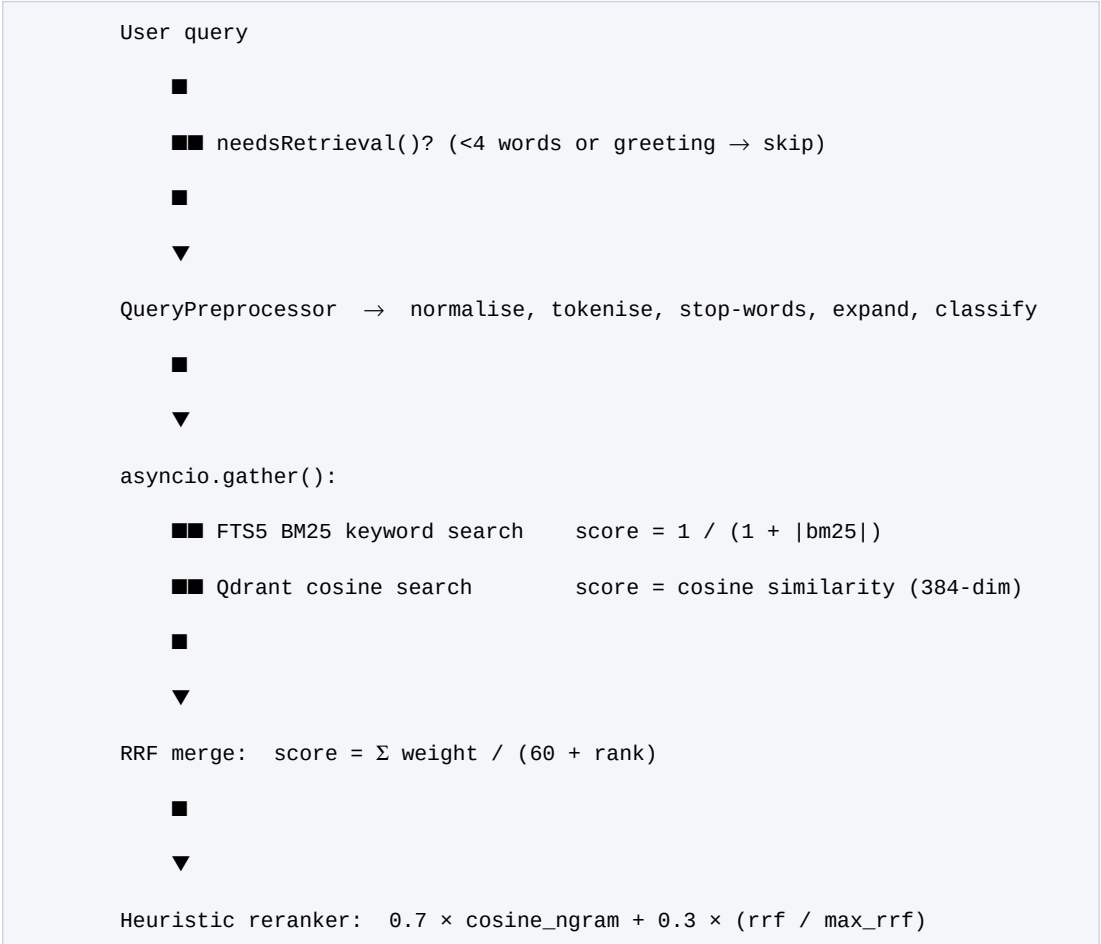
Hybrid Semantic + Keyword Retrieval · v1.0
2026-08-05

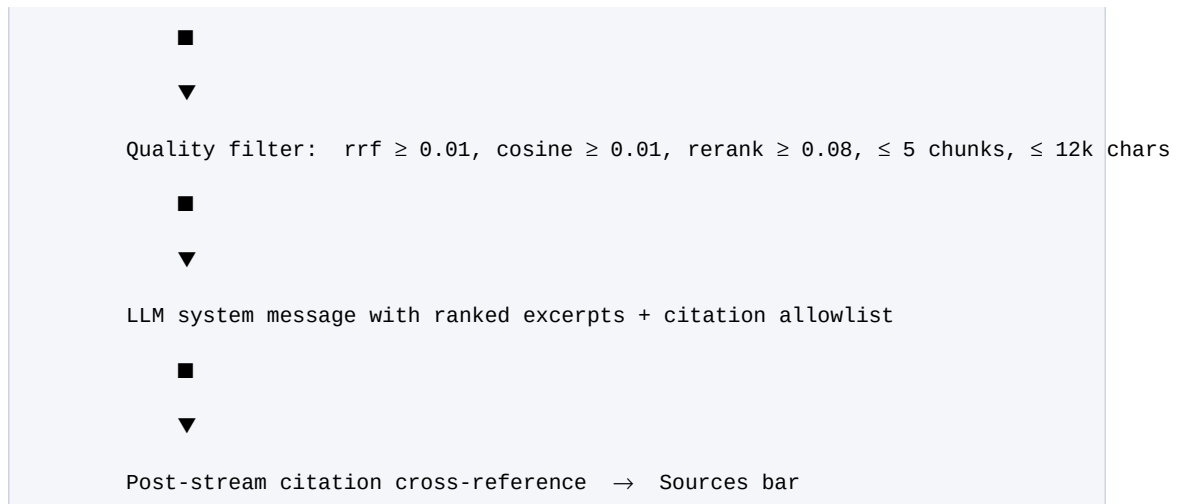
Pipeline Overview

The search pipeline is a seven-stage system that converts a raw user query into a ranked, quality-filtered set of document excerpts that are injected into the LLM context window. No single relevance signal is trusted alone: BM25 keyword search catches exact matches that embeddings miss; cosine similarity catches conceptual matches that BM25 misses; Reciprocal Rank Fusion (RRF) combines their rankings without knowing their absolute scores; and a character-bigram reranker re-scores results by actual query-to-chunk text overlap as a final relevance gate.

Stage	Name	Description
Stage 0	Indexing	File hashing, chunking, embedding, dual write to SQLite + Qdrant
Stage 1	Query Preprocessing	Normalise → tokenise → stop-words → fuzzy flag → expand → intent
Stage 2	Hybrid Search	BM25 keyword (FTS5) and cosine semantic (Qdrant) run in parallel
Stage 3	RRF Merge	Reciprocal Rank Fusion combines both result lists
Stage 4	Heuristic Reranker	Character-bigram cosine re-scores top-20 candidates
Stage 5	Quality Filter	Threshold gates and character budget applied
Stage 6	Citation X-Ref	Post-LLM: sources bar shows only cited documents

End-to-End Flow





Stage 0 — Indexing (Background, One-Time)

Indexing runs once per file, in the background. **file_indexer.py** computes a SHA-256 hash of each file and skips it if the hash matches a previously stored value, making re-indexing incremental and efficient.

Chunking — text_chunker.py

A sliding window of **1,500 characters** with a **200-character overlap** is applied to each document. At every boundary the chunker searches backward within the overlap zone for a sentence break `((?<=[.!?])\s+)` and moves the cut there to avoid splitting mid-sentence. Each chunk is returned as a `(text, char_start, char_end)` tuple.

Embedding — embedding_service.py

The model **BAAI/bge-small-en-v1.5** runs via ONNX/fastembed. It produces a **384-dimensional float vector** for every chunk.

Dual Write — chunk_repository.py

Each chunk and its embedding are written to two stores simultaneously:

- **SQLite chunks table** — stores full text content, chunk index, and character offsets
- **SQLite chunks_fts FTS5 virtual table** — Porter stemmer tokenisation, used for keyword search
- **Qdrant collection document_chunks** — cosine distance, 384 dims, used for semantic search

Stage 1 — Query Preprocessing

query_preprocessor.py runs six sequential steps on the raw user query before any search is performed.

1 — Normalise

Unicode NFC normalisation, lowercase, collapse whitespace.

2 — Tokenise

Split on `[\s,;:!?()[\]{}<>'"]+`.

3 — Stop-word removal

Remove a 70-term English frozenset. Preserves semantically important negations: *no*, *not*, *up*.

4 — Fuzzy flag

Marks tokens with 3+ repeated characters or high consonant density for downstream handling.

5 — Query expansion

Maps 18 abbreviations to synonym lists. Examples: *ai* → [artificial intelligence, machine learning] *rag* → [retrieval augmented generation]

6 — Intent detection

Classifies intent as: COMPARISON > FACTUAL > NAVIGATIONAL > EXPLORATORY. The `search_text` passed downstream is the concatenation of filtered tokens and expanded terms.

Stage 2 — Hybrid Search

`hybrid_orchestrator.py` launches both search providers concurrently via `asyncio.gather()`. Each fetches `top_k × 3` candidates (fetch multiplier = 3) to give the RRF merge stage a wider candidate pool.

2a — Keyword Search (FTS5 / BM25)

`keyword_search.py` wraps each query token in double quotes to prevent FTS5 operator injection:

```
"token1" "token2" "token3"
```

The query runs against the FTS5 table using `bm25(chunks_fts)`. SQLite BM25 returns negative values (lower = better), so scores are normalised to the range (0, 1]:

```
normalised_score = 1.0 / (1.0 + |raw_bm25|)
```

2b — Semantic Search (Qdrant / Cosine)

`qdrant_search.py` generates a 384-dimensional query vector via `EmbeddingService.generate(search_text)` and queries Qdrant using cosine similarity. Hit payloads contain `chunk_id` strings which are then hydrated from SQLite to retrieve full text content.

Stage 3 — RRF Merge

`hybrid_orchestrator.py` merges the two result lists using Reciprocal Rank Fusion (RRF). Each chunk receives a score contribution from every list it appears in:

```
rrf_score(chunk) = keyword_weight / (60 + keyword_rank)
                  + semantic_weight / (60 + semantic_rank)
```

The constant **k = 60** is the standard RRF smoothing value from the literature. It prevents top-ranked results from dominating by compressing the rank differences. Ranks are 1-based (rank 1 = best match). Weights default to 1.0 each and can be tuned per-request.

Numerical Example

A chunk ranked 1st in semantic and 5th in keyword:

```
1/(60+1) + 1/(60+5) = 0.0164 + 0.0154 = 0.0318

vs. a chunk ranked 1st in semantic only:

1/(60+1) = 0.0164
```

A chunk appearing in both lists always outscores a chunk appearing in one list alone — this is why hybrid retrieval outperforms either provider individually.

Stage 4 — Heuristic Reranker

Both the backend (**reranker.py**) and the frontend (**useConversation.ts**) run an identical reranker on the top-20 RRF candidates. The dual implementation ensures consistent ranking whether the result is consumed by the API or directly by the chat interface.

Character Bigram TF Vectorisation

Lowercase word-tokenise each text using `\w+`. For every token, extract all overlapping character bigrams:

```
"hello" → ["he", "el", "ll", "lo"]
```

Count occurrences and divide by the total bigram count to get a TF vector {bigram: frequency}.

Cosine Similarity

```
dot    = Σ a[k] * b[k]    for k in b

norm_a = √(Σ a[v]²)

norm_b = √(Σ b[v]²)

cosine = dot / (norm_a × norm_b)    [0 if denominator = 0]
```

Final Rerank Score

```
rerank_score = 0.7 × cosine(ngramTf(query), ngramTf(chunk_content))
              + 0.3 × (rrf_score / max_rrf_score)
```

The 0.3 position bonus means that a keyword rank-1 hit ($\text{rrf}/\text{max} = 1.0$) receives a free +0.3 contribution to its blended score even if character-level overlap is low. This rewards documents that scored well in the initial retrieval stage while still allowing strong cosine matches to overtake them.

Stage 5 — Quality Filter & Character Budget

`context_assembler.py` (backend) and `useConversation.ts` (frontend) apply the following gates in order. The frontend thresholds are stricter because the LLM call cost is higher — it is better to return nothing than to inject irrelevant context that confuses the model.

Filter	Backend Threshold	Frontend Threshold
<code>rrf_score</code>	≥ 0.004	≥ 0.01
<code>cosine_score</code>	—	≥ 0.01
<code>rerank_score</code>	—	≥ 0.08
<code>Max chunks</code>	5	5
<code>Max characters</code>	12,000	12,000

Stage 6 — Citation Cross-Reference

After the LLM streams its response, `useConversation.ts` extracts all file path references using:

```
/\[([^\]]+\.[a-zA-Z0-9]+)\]/g
```

Only chunks whose `documentPath` appears in the cited set are shown in the Sources bar. This ensures the Sources bar displays documents the model actually used, not everything that passed the retrieval quality filter.

Design Rationale

The key insight behind this architecture is that relevance is multi-dimensional and no single signal captures it reliably:

- **BM25 keyword search** is strong on exact term matches and technical identifiers (function names, error codes, acronyms) but fails on conceptual or paraphrased queries.
- **Dense vector search** handles synonymy and concept proximity well but can miss rare terms that were never seen together in training.
- **RRF** avoids the score normalisation problem by working on ranks rather than raw scores. A document ranked highly by both providers is almost certainly relevant.
- **The bigram reranker** adds a final character-level overlap check that is independent of both BM25 and embedding models, catching cases where the initial retrieval was coarse.
- **Dual filter thresholds** (strict frontend, relaxed backend) allow the API to return more candidates for downstream consumers while protecting LLM call quality in the chat interface.

Module Reference

Module	Layer	Responsibility
file_indexer.py	Infra	File hashing, skip-if-unchanged, orchestrates indexing
text_chunker.py	Infra	Sliding-window chunking with sentence-boundary adjustment
embedding_service.py	Infra	BGE-small-en-v1.5 via ONNX/fastembed → 384-dim vectors
chunk_repository.py	Infra	Dual write to SQLite (chunks + FTS5) and Qdrant
query_preprocessor.py	Domain	Normalise, tokenise, expand, classify intent
hybrid_orchestrator.py	Domain	Parallel search dispatch + RRF merge
keyword_search.py	Infra	BM25 FTS5 query, score normalisation
qdrant_search.py	Infra	Dense vector query, chunk hydration
reranker.py	Domain	Character-bigram cosine reranking (backend)
context_assembler.py	Domain	Quality filter, character budget, context assembly
useConversation.ts	Frontend	Reranker mirror, quality filter, citation cross-ref